

Μεταγλωττιστές 2025

Προγραμματιστική Εργασία #2

Για την επίλυση της εργασίας θα ξεκινήσετε έχοντας ως βάση το εξής αρχείο Python:

<https://mixstef.github.io/courses/compilers/if-while.py>

Το αρχείο αυτό υλοποιεί έναν συντακτικό αναλυτή με κατασκευή ενδιάμεσης αναπαράστασης σε μορφή AST, ακολουθούμενο από έναν διερμηνευτή (interpreter) για τη γραμματική που θα βρείτε στο:

<https://mixstef.github.io/courses/compilers/if-while.pdf>

Η γραμματική υποστηρίζει εντολές ανάθεσης (=), εκτύπωσης (print), ελέγχου ροής (if και if...else) και επανάληψης (while). Ο διερμηνευτής χρησιμοποιεί απλές (scalar) μεταβλητές τύπου float σφαιρικής εμβέλειας (globals) χωρίς δήλωση. Υποστηρίζονται αριθμητικές και λογικές εκφράσεις.

A. Ζητούμενο

Στην προηγούμενη γλώσσα προσθέστε τη δυνατότητα χρήσης μεταβλητών τύπου float array. Η προδιαγραφή υποστήριξης των arrays που πρέπει να υλοποιήσετε είναι η ακόλουθη:

1) Οι αναφορές στα arrays έχουν τη μορφή `var[expr] = ...` (ανάθεση σε στοιχείο array) και `... = ... var[expr] ...` (λήψη τιμής στοιχείου array).

2) Μια μεταβλητή σε συγκεκριμένη στιγμή μπορεί δυναμικά να είναι τύπου float ή τύπου float array, ανάλογα με την πιο πρόσφατη ανάθεση που έχει προηγηθεί:

```
x = 5
// εδώ το x έχει τύπο float
x[4] = 0.99
// εδώ το x έχει τύπο float array
```

3) Σε κάθε ανάθεση που αλλάζει τύπο, η/οι προηγούμενη/ες τιμή/ές της μεταβλητής διαγράφονται.

4) Τα arrays λειτουργούν ως «αραιοί» (sparse) πίνακες: δεν υπάρχει προκαταρκτική δέσμευση χώρου και η ανάθεση στο στοιχείο `a[i]` δεν προϋποθέτει την ύπαρξη των στοιχείων `a[x]` με `x < i` πριν την ανάθεση ούτε συνεπάγεται την ύπαρξη των στοιχείων `a[x]` μετά την ανάθεση.

5) Η λήψη τιμής με τύπο μεταβλητής διαφορετικό από τον τρέχοντα προκαλεί σφάλμα εκτέλεσης (RunError). Παραδείγματα:

```
x = 5
// εδώ το x έχει τύπο float
print x[4] // σφάλμα!
x[4] = 0.99
// εδώ το x έχει τύπο float array
print x    // σφάλμα!
```

6) Η λήψη τιμής από στοιχείο ενός array που δεν έχει τεθεί σε κάποια τιμή προηγουμένως προκαλεί επίσης σφάλμα εκτέλεσης (RunError).

7) Σε ανάθεση ή λήψη τιμής σε/από στοιχείο array `var[expr]`, η έκφραση `expr` πρέπει να έχει θετική τιμή (≥ 0) με κλασματικό μέρος ίσο με το 0. Σε αντίθετη περίπτωση προκαλείται σφάλμα εκτέλεσης (RunError).

```
x[4] = 0.99    // έγκυρο  
x[4.0] = 0.99  // έγκυρο  
x[4.23] = 0.99 // σφάλμα!
```

B. Διαδικασία επίλυσης

Ακολουθήστε τα παρακάτω βήματα για τη δημιουργία του κώδικα υλοποίησης σε Python. Οι ερωτήσεις (με αριθμηση E.x.y) που τίθενται σε κάθε στάδιο **θα πρέπει να απαντηθούν στη συνοδευτική αναφορά που θα παραδώσετε μαζί με τον κώδικα.**

1. Τροποποίηση γραμματικής

Τροποποιήστε την υπάρχουσα γραμματική έτσι ώστε να συμπεριλάβετε τους κανόνες για τη συντακτική ανάλυση των arrays.

(E.1.1) Ποιοι νέοι κανόνες της γραμματικής εισάγονται; Περιγράψτε τους νέους κανόνες και τον λόγο που προστίθενται.

(E.1.2) Ποιοι υπάρχοντες κανόνες τροποποιούνται; Περιγράψτε τις αλλαγές στους κανόνες αυτούς και τον σκοπό τους.

(E.1.3) Ποια νέα τερματικά και μη τερματικά σύμβολα εισάγονται;

(E.1.4) Δώστε τον πίνακα με τα σύνολα FIRST και FOLLOW (τα τελευταία μόνο όταν υπάρχει το ε στο FIRST). Υπογραμμίστε τα νέα στοιχεία στα σύνολα FIRST και FOLLOW σε σχέση με εκείνα της αρχικής γραμματικής.

2. Τροποποίηση κώδικα συντακτικού αναλυτή

Με βάση τις απαντήσεις σας στο (1) προσθέστε τον κατάλληλο κώδικα για να υλοποιήσετε τα νέα μη τερματικά σύμβολα και για συμπεριλάβετε τις αλλαγές στα ήδη υπάρχοντα μη τερματικά σύμβολα. **Προσοχή: θα πρέπει να γίνουν μόνο οι απολύτως απαραίτητες προσθήκες ή τροποποιήσεις.**

(E.2.1) Περιγράψτε συνοπτικά τον κώδικα για την υλοποίηση κάθε νέου μη τερματικού συμβόλου.

(E.2.2) Περιγράψτε κάθε μία αλλαγή που κάνατε στις μεθόδους των υπαρχόντων μη τερματικών συμβόλων και τον σκοπό της.

(E.2.3) Περιγράψτε τις προσθήκες στον Tokenizer.

Στο σημείο αυτό δοκιμάστε την ορθή συντακτική ανάλυση του εξής πηγαίου κώδικα:

```
a = 2 + 7.55*44
print a
if a-7!=3 or a<355 {
    b[8] = 3*(a-99.01)
    it = 5
    while it>0 {
        c[it] = it+b[8]*0.23
        print c[it+5-3-2]
        it = it - 1
    }
}
else
    print a-3.14
```

3. Προσθήκη κώδικα κατασκευής AST

Στη συνέχεια προσθέστε/τροποποιήστε τον κώδικα του συντακτικού αναλυτή για τη δημιουργία του νέου AST.

(E.3.1) Περιγράψτε τα νέα είδη ASTNodes που απαιτούνται για την υποστήριξη των arrays. Για κάθε νέο είδος ASTNode παραθέστε τον τύπο, τα attributes και subnodes αυτού. Τέλος, περιγράψτε πώς χρησιμοποιείται κάθε νέο είδος ASTNode.

(E.3.2) Προσδιορίστε στον κώδικά σας τα σημεία δημιουργίας κάθε νέου είδους ASTNode, και τι επιστρέφει η συνάρτηση κάθε νέου μη τερματικού συμβόλου (βλ. E.2.1).

4. Προσθήκη κώδικα διερμηνευτή (interpreter)

Ως τελευταίο βήμα προσθέστε στον κώδικα του διερμηνευτή τις λειτουργίες υποστήριξης των arrays σύμφωνα με την προδιαγραφή της γλώσσας (βλ. ενότητα A. Ζητούμενο).

(E.4.1) Σχεδιάστε τον τρόπο αποθήκευσης των arrays. Με ποια δομή της Python θα τον υλοποιήσετε; Πώς θα γίνεται η διάκριση μεταξύ float και float array;

(E.4.2) Παραθέστε τα σημεία του κώδικα όπου υλοποιούνται οι κανόνες 2 έως 7 (βλ. ενότητα A. Ζητούμενο) της προδιαγραφής της γλώσσας για τα arrays.

Η ανάλυση και διερμηνεία του δοκιμαστικού πηγαίου κώδικα θα πρέπει να τυπώνει τα εξής αποτελέσματα:

```
334.2
167.28109999999998
166.28109999999998
165.28109999999998
164.28109999999998
163.28109999999998
```

Βεβαιωθείτε επίσης ότι κάθε μία περίπτωση σφάλματος από τα επόμενα εντοπίζεται κατά την εκτέλεση:

```
x[18] = 0.33
print x          // σφάλμα! το x είναι array
print x[3]       // σφάλμα! το x[3] δεν έχει οριστεί
x[5-6] = 0.1     // σφάλμα! άκυρη τιμή index
x[3.14] = 7      // σφάλμα! άκυρη τιμή index
x = 23
print x[45]      // σφάλμα! το x είναι scalar
```

Γ. Παραδοτέο

i) Όταν έχετε ολοκληρώσει την επίλυση της εργασίας, περάστε τον τελικό κώδικα σε ένα jupyter notebook (αν δεν χρησιμοποιείτε notebook από την αρχή). Εκτελέστε τον συνολικό κώδικα με τον δοκιμαστικό πηγαίο κώδικα. Εκτελέστε επίσης τα παραδείγματα σφαλμάτων. **Τα αποτελέσματα των δοκιμών θα πρέπει να εκτυπωθούν και να φαίνονται στο τελικό jupyter notebook που θα παραδώσετε.**

ii) Ετοιμάστε αναφορά σε μορφή pdf με τις απαντήσεις σας στα ερωτήματα E.x.y των προηγούμενων ενοτήτων.

Ανεβάστε το τελικό σας notebook και την αναφορά pdf σε ένα μοναδικό αρχείο zip στο opencourses (**Εργασία 2**) έως και τη **Δευτέρα 2/6**.

Βαθμολόγηση εργασίας

Κριτήριο	% βαθμού
Επιτυχής συγγραφή κώδικα με ορθή εμφάνιση αποτελεσμάτων	40%
Πλήρεις και τεκμηριωμένες απαντήσεις των ερωτημάτων στην αναφορά	40%
Κώδικας χωρίς περιττά στοιχεία, μόνο με τις απολύτως αναγκαίες προσθήκες	20%